```
================================================================================
题 3 - 第一步：SPI Flash 驱动跑通（读 ID）
板子：野火 STM32F103ZET6（霸道）  板载 W25Q64，8MB
工程：Exercise03/flash   （CubeMX 新建的干净工程）
说明：这是照着自己敲的参考稿，不是编译文件（_ 前缀 + .txt，Keil 不会碰它）

注意：Exercise02/Menu02/Interface/FLASH 里那份同名稿子作废。
      那份是"路线 A"（往老工程里手工塞 HAL SPI 文件），
      你现在新建了 CubeMX 工程，"路线 B"走成了，那些步骤全都不需要了。
================================================================================
```

【0】这一步要拿到什么

目标只有一个：串口打印出

```
    FLASH JEDEC = 0xEF4017
```

0xEF4017 = Winbond(0xEF) + 0x4017，也就是 W25Q64 / 8MB。
读到这个值 = SPI2 的时钟、数据、片选三根线全通、芯片活着。
读不到就先别往下走，第二步的擦写只会更乱。

这个工程现在已经有什么（CubeMX 生成的，别重复做）：

```
  [OK] SPI2 已初始化  PB13=SCK  PB14=MISO  PB15=MOSI
                  主机 / 8bit / CPOL=0 / CPHA=0（模式 0）/ NSS=软件 / 36MHz÷2 = 18MHz
  [OK] HAL_SPI_MODULE_ENABLED 已打开（stm32f1xx_hal_conf.h 第 65 行）
  [OK] stm32f1xx_hal_spi.c 已经在 Keil 工程分组里
  [OK] USART1 已初始化  PA9=TX  PA10=RX  115200-8-N-1
  [OK] MicroLIB 已勾选（这很关键，printf 能不能用全看它）
```

现在缺三样（就是你要干的活）：

```
  [NG] PB12（片选）没配成输出  ← 最容易漏，漏了就是读回 0xFFFFFF
  [NG] Interface/FLASH/spi_flash.c / .h 还没写
  [NG] printf 没重定向到 USART1
```

【1】硬件映射（板上焊死的，别改）

信号	引脚	MCU 侧配置
CS	PB12	普通推挽输出，空闲为高（软件片选）
SCK	PB13	SPI2_SCK，复用推挽
MISO	PB14	输入（浮空）
MOSI	PB15	SPI2_MOSI，复用推挽

SPI2 挂在 APB1 上。这个工程 APB1 = 36MHz，
所以 SPI 波特率 = 36MHz ÷ 分频，没有别的来源。
现在配的是 2 分频 = 18MHz。

JEDEC ID 对照（0x9F 读回 3 个字节：厂商 + 容量高 + 容量低）

```
  0xEF4014 -> W25Q80    1MB
  0xEF4015 -> W25Q16    2MB
  0xEF4016 -> W25Q32    4MB
  0xEF4017 -> W25Q64    8MB    <-- 霸道板载这颗，你要读到它
```

0xEF4018 -> W25Q128 16MB
厂商字节 0xEF = Winbond（华邦）

【2】为什么第一步只读 ID 就够（先想明白再敲）

读 JEDEC ID 这条指令 0x9F 是"免前置条件"的：
- 不用先擦除
- 不用先发写使能
- 不用给地址

上电稳定后拉低 CS，发 0x9F，跟着收 3 个字节，拉高 CS，完事。
所以它是整个驱动里最便宜的自检手段 —— 硬件有问题第一时间就暴露。

顺便把 SPI 的脾气定死，后面全靠这个理解：
SPI 是"主机发时钟，从机跟着吐数据"的全双工总线。
你想收 1 个字节，就必须同时发 1 个字节出去 —— 发什么无所谓（习惯发 0xFF）。
不发就没有时钟；没有时钟，从机一个 bit 都不会给。
这就是为什么 ReadID 里写了两次"发 0xFF"：那不是给芯片的命令，
那是"借发数据产生时钟，好让芯片把 ID 吐出来"。

【3】动手顺序（一共 6 步，前 2 步必须先做完）

--- 第 1 步：CubeMX 里配 PB12（先做这个，做完再写代码）---
打开 MDK-ARM 同级的 flash.ioc（在 flash/ 目录下）
-> 左边 Pinout view 里点 PB12 -> 选 GPIO_Output
-> 右侧 System Core / GPIO -> 选中 PB12，设：

```
GPIO output level      : High      ← 一定要 High，空闲不能选中芯片
GPIO mode              : Output Push Pull
GPIO Pull-up/Pull-down : No pull-up and no pull-down
Maximum output speed   : High
User Label             : FLASH_CS  ← 打了标签会生成 FLASH_CS_Pin 宏，代码好读
```

-> 点 GENERATE CODE

为什么必须走 CubeMX 而不是手改 gpio.c：
MX_GPIO_Init() 函数体里没有 USER CODE 区，手写进去的内容
下次重新生成会被整段覆盖。这题后面还要加外设，一定会再生成。
（配完以后 gpio.c 里会多出 PB12 的初始化，和 FLASH_CS_Pin 宏在 main.h）

检查一下：Core/Inc/main.h 末尾应该出现

```
#define FLASH_CS_Pin       GPIO_PIN_12
#define FLASH_CS_GPIO_Port GPIOB
```

没有就说明没配成功，回去重来。

--- 第 2 步：printf 重定向到 USART1 ---
文件：Core/Src/usart.c
位置 1：顶部的 /* USER CODE BEGIN 0 */ 区，加一行

```
#include <stdio.h>
```

位置 2：文件末尾的 /* USER CODE BEGIN 1 */ 区，加

```
/* printf 最终会调到 fputc，这里把它改成从 USART1 发出去 */
int fputc(int ch, FILE *f)
{
  HAL_UART_Transmit(&huart1, (uint8_t *)&ch, 1, 0xFFFF);
```

```
    return ch;
}
```

前提：Options for Target -> Target 里 "Use MicroLIB" 必须勾着。
      这个工程已经是勾着的，别去动它。
      没勾的话 printf 会进半主机模式，程序卡在 BKPT 不动，
      表现就是"灯不闪、串口一个字都没有"。

--- 第 3 步：写 Interface/FLASH/spi_flash.h（内容见【4.1】）---

--- 第 4 步：写 Interface/FLASH/spi_flash.c（内容见【4.2】）---
先只敲 SPI_TxRx + SPI_Flash_Init + SPI_Flash_ReadID 三个，
编译过一遍，确认没红叉，再补 ReadSR1 / IsBusy。
一次全敲完再编译，报错会堆在一起看不出根因。

--- 第 5 步：Keil 里挂文件（最容易漏的一步）---
1) Project 窗口 -> 右键 Target 名 -> Manage Project Items
     -> Groups 里 New 一个 "Interface/FLASH"
     -> 选中它，Files 里 Add Files 选 spi_flash.c
2) Options for Target -> C/C++ -> Include Paths 末尾加一行
      ..¥Interface¥FLASH
少 2) 的表现：编译报 "cannot open source input file spi_flash.h"，
            或者 spi_flash.h 上有红波浪线但编译能过（IntelliSense 假警报）。

--- 第 6 步：main.c 加成调用（改法见【4.3】）---

--- 编译 + 下载 + 开串口 ---
串口参数 115200-8-N-1，VOFA+ 或任意串口助手都行。
下载完按一下复位，看输出。

【4】代码

【4.1】Interface/FLASH/spi_flash.h

```
#ifndef __SPI_FLASH_H
#define __SPI_FLASH_H

#include "main.h"

/* W25Q 系列指令（手册 Instruction Set 那一章，第一步只用这三个）*/
#define W25Q_CMD_JEDEC_ID       0x9F   /* 读厂商 + 容量 ID，回 3 字节 */
#define W25Q_CMD_READ_SR1       0x05   /* 读状态寄存器 1 */
#define W25Q_CMD_WRITE_ENABLE   0x06   /* 写使能，第二步擦/写之前要用 */
#define W25Q_CMD_RELEASE_PD     0xAB   /* 退出掉电模式，芯片睡死了才用 */

void     SPI_Flash_Init(void);
uint32_t SPI_Flash_ReadID(void);
uint8_t  SPI_Flash_ReadSR1(void);
uint8_t  SPI_Flash_IsBusy(void);

#endif /* __SPI_FLASH_H */
```

【4.2】Interface/FLASH/spi_flash.c

```
#include "spi_flash.h"
#include "spi.h"          /* 为了拿 hspi2 */

/* 片选：只有这一处，以后换引脚只改这两行 */
#define FLASH_CS_LOW()    HAL_GPIO_WritePin(FLASH_CS_GPIO_Port, FLASH_CS_Pin, GPIO_PIN_RESET)
#define FLASH_CS_HIGH()   HAL_GPIO_WritePin(FLASH_CS_GPIO_Port, FLASH_CS_Pin, GPIO_PIN_SET)

/* ------------------------------------------------------------------
   底层：全双工交换 1 个字节
   注意它"既发又收"—— SPI 没有纯读，发出去的字节就是为了换时钟。
   超时返回 0xFF 是故意的：让上层读到 0xFFFFFF，一眼看出是通信问题，
   而不是悄悄返回 0 让你以为是"芯片真给了 0"。
   ------------------------------------------------------------------ */
static uint8_t SPI_TxRx(uint8_t tx)
{
  uint8_t rx = 0xFF;

  if (HAL_SPI_TransmitReceive(&hspi2, &tx, &rx, 1, 100) != HAL_OK)
  {
    return 0xFF;
  }
  return rx;
}

/* 初始化：片选拉回空闲高，等芯片上电稳定（W25Q 上电到可操作约 5ms）*/
void SPI_Flash_Init(void)
{
  FLASH_CS_HIGH();
  HAL_Delay(10);
}

/* ------------------------------------------------------------------
   读 JEDEC ID：拉低 CS -> 发 0x9F -> 收 3 字节 -> 拉高 CS
   收的时候发的 0xFF 是 dummy，只为产生时钟
   ------------------------------------------------------------------ */
uint32_t SPI_Flash_ReadID(void)
{
  uint32_t id = 0;

  FLASH_CS_LOW();

  SPI_TxRx(W25Q_CMD_JEDEC_ID);
  id  = (uint32_t)SPI_TxRx(0xFF) << 16;   /* 厂商 ID，应为 0xEF */
  id |= (uint32_t)SPI_TxRx(0xFF) << 8;    /* 容量高字节，0x40 */
  id |= (uint32_t)SPI_TxRx(0xFF);         /* 容量低字节，0x17 = 64Mbit */

  FLASH_CS_HIGH();

  return id;
}
```

```
/* 读状态寄存器 1：bit0 = BUSY，bit1 = WEL（写使能锁存）*/
uint8_t SPI_Flash_ReadSR1(void)
{
  uint8_t sr;

  FLASH_CS_LOW();
  SPI_TxRx(W25Q_CMD_READ_SR1);
  sr = SPI_TxRx(0xFF);
  FLASH_CS_HIGH();

  return sr;
}

/* BUSY = 1 说明正在擦/写，这期间它不接任何新指令 */
uint8_t SPI_Flash_IsBusy(void)
{
  return (uint8_t)(SPI_Flash_ReadSR1() & 0x01);
}
```

【4.3】Core/Src/main.c 的改动（三处，全在 USER CODE 里）

---

1) /* USER CODE BEGIN Includes */ 区加：

```
#include <stdio.h>
#include "spi_flash.h"
```

2) /* USER CODE BEGIN 2 */ 区（就在 MX_USART1_UART_Init() 后面）加：

```
printf("¥r¥n=== SPI Flash bring-up ===¥r¥n");

SPI_Flash_Init();

{
  uint32_t id  = SPI_Flash_ReadID();
  uint8_t  sr1 = SPI_Flash_ReadSR1();

  printf("FLASH JEDEC = 0x%06X¥r¥n", (unsigned int)id);
  printf("FLASH SR1   = 0x%02X¥r¥n", (unsigned int)sr1);

  if (id == 0xEF4017)
  {
    printf("OK : W25Q64 / 8MB, SPI2 works¥r¥n");
  }
  else if (id == 0xFFFFFF)
  {
    printf("NG : MISO 一直是高 -> CS 没拉低 / 芯片没供电 / SPI2 时钟没开¥r¥n");
  }
  else if (id == 0x000000)
  {
    printf("NG : MISO 一直是低 -> MISO 短地 / 芯片没焊上 / 片选提前拉高¥r¥n");
  }
  else
  {
    printf("NG : ID 认出来了但不是 W25Q64, 查芯片型号¥r¥n");
```

```
    }
  }
```

3) 不用改别的。MX_SPI2_Init() 已经排在 MX_USART1_UART_Init() 前面了。

两个细节：
- 打印用 %06X 不用 %lX：ID 只有 24 位，转成 unsigned int 打 %X 最省事。
  Keil 的 printf 对 %l 的支持要看编译器版本，有时候会打出 "lX" 或者乱码。
- 全部用英文。Keil 源文件是 GBK，串口助手按 UTF-8 显示，
  中文必乱码（你 ex08 里那个 "乱码" 就是这么来的）。
  [!] 上面 printf 里那几行中文说明是给你看的，敲的时候换成英文或直接删掉。

【5】怎么算通过

串口应该看到：

```
=== SPI Flash bring-up ===
FLASH JEDEC = 0xEF4017
FLASH SR1   = 0x00
OK : W25Q64 / 8MB, SPI2 works
```

0xEF4017 -> 型号对上了，说明 SCK/MOSI/MISO/CS 四根线全通。
SR1 = 0x00 -> 芯片空闲（BUSY=0），并且 WEL=0（从没发过写使能，正常）。

【6】读到怪值时先查哪儿（照这个顺序，别乱试）

读到的值	说明什么	先查
0x000000	MISO 一直低	CS 有没有真的拉低 / MISO 是否短地 / 芯片是不是没焊上
0xFFFFFF	MISO 一直高	芯片没供电 / CS 没接 / SPI2 时钟没开 / PB12 是不是漏配成输出了
0x0000FF	只收到一个字节	片选提前拉高，或者循环次数写错
0xEF40xx	厂商对、容量不对	查芯片型号，可能不是 W25Q64
0xEF4017 但偶发错一次 0xFFFFFF 一次正常	速率临界	CubeMX 里分频从 2 改成 8（4.5MHz）再试 多半是 CS 拉高后太快又拉低， 或者在 __HAL_SPI_ENABLE 之前就去读了

终极判断法：拿示波器/逻辑分析仪看 SCK。
  发指令时应该正好 8 个脉冲（读 ID 全过程 4×8 = 32 个脉冲）。
  一个脉冲都没有 = SPI2 时钟没开，或者 HAL_SPI_Init 根本没被调用。
  有脉冲但 MISO 不动 = 从机侧的事（供电/CS/MISO 走线）。

【7】这一步最容易踩的坑（按踩到概率排序）

1. PB12 忘了配。NSS=SOFT 之后 HAL 不会帮你拉 CS，
   片选永远是浮空/低电平，芯片根本不理你，读回 0xFFFFFF。
   这是本题第一号杀手。

2. 用 HAL_SPI_Transmit 去读 ID。只发不收，rx 永远是 0。
   读数据必须 HAL_SPI_TransmitReceive（或者 Transmit 完再 Receive，

但那样两次调用中间 CS 会断，多字节读一定错位）。

3. 每读一个字节都要先发一个 0xFF。不发就没有时钟，
芯片不会吐数据，收到的是上一个字节的残留。

4. PB14(MISO) 配成 AF_PP。MISO 是输入，写成输出会两边同时驱动，
波形打架，读回来全是随机值。CubeMX 生成的 GPIO_MODE_INPUT 是对的，
别自作聪明去改。

5. MicroLIB 没勾。printf 进半主机模式，程序卡死在 BKPT。
表现：程序像没跑一样，串口一个字节都没有，Debug 停下来看 PC 在 0x0800_xxxx 附近。

6. Include Paths 里漏了 ..¥Interface¥FLASH。
编译报 "cannot open source input file spi_flash.h"。

7. 手改 gpio.c 而不走 CubeMX。下次 GENERATE CODE 全没，
你会莫名其妙"昨天还好好的"。

8. printf 打中文。见【4.3】末尾。

9. 一开始就怀疑芯片坏了。0xFFFFFF 十次有九次是片选或时钟，
先量 CS 有没有动，再怀疑芯片。

10. HAL_SPI_Init 失败会进 Error_Handler()，里面是 __disable_irq() + while(1)，
死得无声无息。如果串口什么都没打印，先在 Error_Handler 里打个断点看看有没有进去。

【8】动手打勾（照着顺序，别跳）

[ ] CubeMX 打开 flash.ioc，配 PB12 = GPIO_Output / High / 推挽 / High speed / Label=FLASH_CS
[ ] GENERATE CODE，确认 main.h 里出现 FLASH_CS_Pin / FLASH_CS_GPIO_Port
[ ] usart.c：加 #include <stdio.h> 和 fputc 重定向
[ ] 编译一次（这时候应该 0 Error，先确认工程本身是好的）
[ ] 写 Interface/FLASH/spi_flash.h
[ ] 写 Interface/FLASH/spi_flash.c（先三个函数，编译过再补后两个）
[ ] Keil 新建分组 Interface/FLASH，挂 spi_flash.c
[ ] Include Paths 加 ..¥Interface¥FLASH
[ ] main.c 加 include + 初始化 + 打印
[ ] 编译下载，复位，串口看到 0xEF4017
[ ] 故意把 PB12 那条 CS 宏注释掉再跑一次，看看 0xFFFFFF 长什么样（强烈建议做，认坑）

最后那条不是折腾你 —— 见过一次坏的样子，以后半夜调试能省两小时。

【9】第二步会用到什么（先有个印象，别现在就写）

- 0x06 WriteEnable：每次擦/写之前单独发一次，拉高 CS 才算生效
- 0x05 读 SR1 轮询 BUSY：擦除要几十毫秒，不能只读一次
- 0x20 扇区擦除（4KB）：24 位地址，发完拉高 CS 才真正开始擦
- 0x02 页编程（256 字节一页）：跨页会回卷到页首，这是最容易错的地方
- 0x03 读数据：地址自动递增，可以连续读
- 到那时候 SPI_TxRx 得升级成"整块收发"版本
（一次 HAL_SPI_TransmitReceive 传一整段，不然单字节调用开销会把速度拖死）

这一步先把 ID 读对，别提前想第二步。

==========